

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	<i>Apply advanced methodologies and agile project management techniques to manage software projects effectively</i>		
Student Name:	Tharika S	Reg. No.:	192521140
Date:	03-10-2026	Duration:	60 minutes

Code of Conduct

I Tharika S / 192521140 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Tharika S

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

- 1. Understand the Problem Statement**
 - Explain the problem in your own words.
 - Identify the objectives of the proposed system.
 - State the scope of the problem.
- 2. Identify Key Stakeholders and Challenges**
 - Identify all the stakeholders involved in the system.
 - Explain the roles and responsibilities of each stakeholder.
 - Discuss the major challenges and issues faced by the stakeholders.
- 3. Analyze the Current Approach**
 - Describe the existing system or current process.
 - Identify its strengths and limitations.
 - Analyze the problems or gaps that need to be addressed.
- 4. Propose a Suitable Solution Using Software Engineering Principles**
 - Design a software-based solution for the given problem.
 - Describe the major functional modules of the proposed system.
 - Identify the functional and non-functional requirements.
 - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
- 5. Justify the Chosen Methodology/Framework**

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
 - Justify why the selected methodology/framework is suitable for the proposed system.
 - Explain how it supports successful project development and maintenance.
- 6. Discuss Expected Outcomes and Limitations**
- Describe the expected benefits of the proposed solution.
 - Explain how the solution addresses the identified challenges.
 - Discuss potential risks, implementation challenges, and limitations.
 - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework	Selects the most suitable	Suitable methodology	Methodology selected but	Outcomes, limitations, or

Justification (10%)	SDLC/model/frame work and provides strong technical justification.	selected with adequate justification.	justification is weak.	future enhancements are missing; report lacks organization and clarity.
---------------------	--	---------------------------------------	------------------------	---

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

Problem Statement - Predictive Analytics System for Early Detection of Land Acquisition Delays

1. Understanding the Problem Statement

1.1 Explain the problem in your own words.

Large public projects such as highways, metro lines, railway tracks, airports and irrigation canals cannot start without land. This land usually belongs to hundreds or even thousands of private owners, so the government has to go through a long chain of steps before it can take it over. The steps include issuing a notification, surveying the land, checking ownership records, valuing the land, deciding the compensation, hearing objections and finally taking possession.

In practice, something goes wrong at one of these steps very often. Records may not match, owners may disagree with the compensation amount, a court case may be filed, or a clearance may not arrive on time. The real trouble is that these delays are usually noticed only when the project is already late. By then the cost has gone up, the contractor is sitting idle and the public is still waiting for the facility.

So the problem is that there is no system which looks at the data of a land acquisition project while it is running and warns the officers early that a particular parcel, village or stage is likely to get delayed. The proposed Predictive Analytics System tries to solve exactly this.

1.2 Identifying the objectives of the proposed system.

The main aim of the system is to predict land acquisition delays before they actually happen so that officers can act in time. The specific objectives are:

- To collect land acquisition data from different departments in one common place.
- To study past projects and find the factors which usually cause delays.
- To predict the delay risk of every land parcel and every stage of the process.
- To send early alerts to the responsible officers when the risk crosses a limit.
- To show the status and risk of all projects on a simple dashboard for quick decisions.
- To reduce cost overruns and improve transparency in the acquisition process.

1.3 The scope of the problem.

The system will cover the stages of land acquisition from the initial notification up to the handing over of possession to the project agency. It will work on data such as land records, survey details, ownership and legal status, court cases, compensation and payment status, and the approval history of each parcel. It will be used by government officers and project agencies at district and state level.

The system will not handle the actual legal decisions, the payment of compensation or the physical survey work. It only analyses the data and gives predictions and alerts. The first version is planned for a single project type (linear infrastructure like roads or rail) and can be extended later.

2. Identifying Key Stakeholders and Challenges

2.1 Identifying all the stakeholders involved in the system.

Many people and departments are connected to land acquisition, so the stakeholders are grouped as follows:

- Land Acquisition Officer / Special Deputy Collector
- Project Manager of the project agency (for example NHAI, Metro Rail or Railways)
- Revenue Department and Survey Department staff
- Land owners and affected families
- Legal department and courts
- Finance department (compensation and payments)
- Senior government authorities such as District Collector and Secretaries
- Contractors and the software development / IT team

2.2 Explanation of the roles and responsibilities of each stakeholder.

Stakeholder	Role and Responsibility
Land Acquisition Officer	Runs the acquisition process, issues notices, checks documents, and updates the status of each parcel in the system.
Project Manager	Tracks project schedule, depends on land availability, and uses the predictions to plan work and manage contractors.
Revenue and Survey Staff	Maintain land records, carry out field surveys and provide correct measurements and ownership details.
Land Owners	Provide ownership proof, accept or object to compensation, and hand over land after payment.
Legal Department / Courts	Handle objections and disputes, give legal opinions and decide court cases related to the land.
Finance Department	Approve funds, calculate compensation and release payments to owners.
Senior Authorities	Monitor overall progress, review high-risk projects and take policy or escalation decisions.
Contractors	Start construction once land is handed over; their idle time and claims depend on delays.
IT / Development Team	Build, deploy and maintain the system, and keep the data and models up to date.

2.3 The major challenges and issues faced by the stakeholders.

Each stakeholder faces different problems, but most of them come from the same root cause, which is the lack of timely and connected information. Officers do not get a clear picture of which parcels are at risk. Project managers cannot plan the work because they do not know when the land will actually be free. Land owners complain about delay in compensation and lack of information about their case. Courts and the legal team are overloaded, so disputes stay pending for years.

The major challenges are listed below.

- **Poor data quality:** Land records are often old, incomplete or do not match the field situation.
- **Scattered data:** Revenue, survey, legal and finance departments keep their own records and do not share them easily.
- **Compensation disputes:** Owners reject the offered amount and go to court, which stops the process for that parcel.
- **Delayed approvals:** Environmental, forest or railway clearances take longer than expected.
- **Late detection:** Problems are found only after the deadline is missed.
- **Cost overruns:** Delay increases project cost through contractor claims and rising land prices.

3. Analysing the Current Approach

3.1 The existing system or current process.

At present, most land acquisition offices follow a manual or semi-manual process. Details of each parcel are kept in files, registers and Excel sheets. Every department maintains its own copy. Progress is checked through periodic review meetings where officers present status reports, and the project manager follows up by phone, letters and emails. Some states have land record portals, but these mostly store records and do not analyse them or predict anything.

Delays are generally handled in a reactive way. When a parcel is stuck, the officer tries to solve it after it has been reported. There is no standard method to find out in advance which parcels will become a problem.

3.2 Identifying its strengths and limitations.

The existing approach has some strengths. It is simple, officers are used to it, and it needs no special software or training. Local officers also have good field knowledge and personal contact with villagers, which helps in solving many problems. Digitised land record portals have improved the availability of basic records.

However, the limitations are more serious:

- Information is manual and often outdated by the time it reaches decision makers.
- No early warning is available; the system only reports what has already happened.
- Data is kept separately by each department, so it is hard to see the full picture.
- Decisions depend on individual experience instead of data and past trends.
- Chances of errors and duplicate entries are high in spreadsheets and files.
- Reports take a lot of time to prepare and do not give a quick project-wise view.

3.3 The problems or gaps that need to be addressed.

When the strengths and limitations are compared, a few clear gaps can be seen. The first gap is the absence of prediction. Past project data holds useful patterns, for example that parcels with unclear ownership and pending court cases are delayed more often, but nobody uses it. The second gap is the missing integration of data from different departments. The third gap is the lack of automatic alerts and priority ranking, so officers spend equal effort on all parcels instead of focusing on risky ones. The fourth gap is poor transparency, because there is no single dashboard that senior authorities can use to track progress. The proposed system is designed to fill these four gaps.

4. Suitable Solution Using Software Engineering Principles

4.1 Designing a software based solution for the given problem.

The proposed solution is a web-based Predictive Analytics System that collects land acquisition data from different sources, cleans and stores it in a central database, and then uses a machine learning model to predict the delay risk of each parcel. The predicted risk is converted into a simple score (Low, Medium or High) and shown on a dashboard. When a parcel crosses the High level, the system sends an alert to the concerned officer.

The architecture follows a layered design so that each layer has a clear job. Data flows from the sources at the top, through ingestion, storage and feature preparation, to the prediction engine, and finally to the users through the dashboard. A separate login and access control block protects the whole system. The block diagram is given below.

In this architecture, the data sources are the existing records from departments. The ingestion module pulls this data (through files, APIs or manual upload), removes duplicates and fixes format problems. The cleaned data is stored in the central database. The feature engineering module then converts raw data into useful inputs for the model, such as number of owners per parcel, days spent at each stage, number of pending objections and past delay history. The prediction engine uses these inputs to calculate the probability of delay. The risk scoring and alert modules use this output, and the dashboard presents it to the project manager, land officer and senior authority.

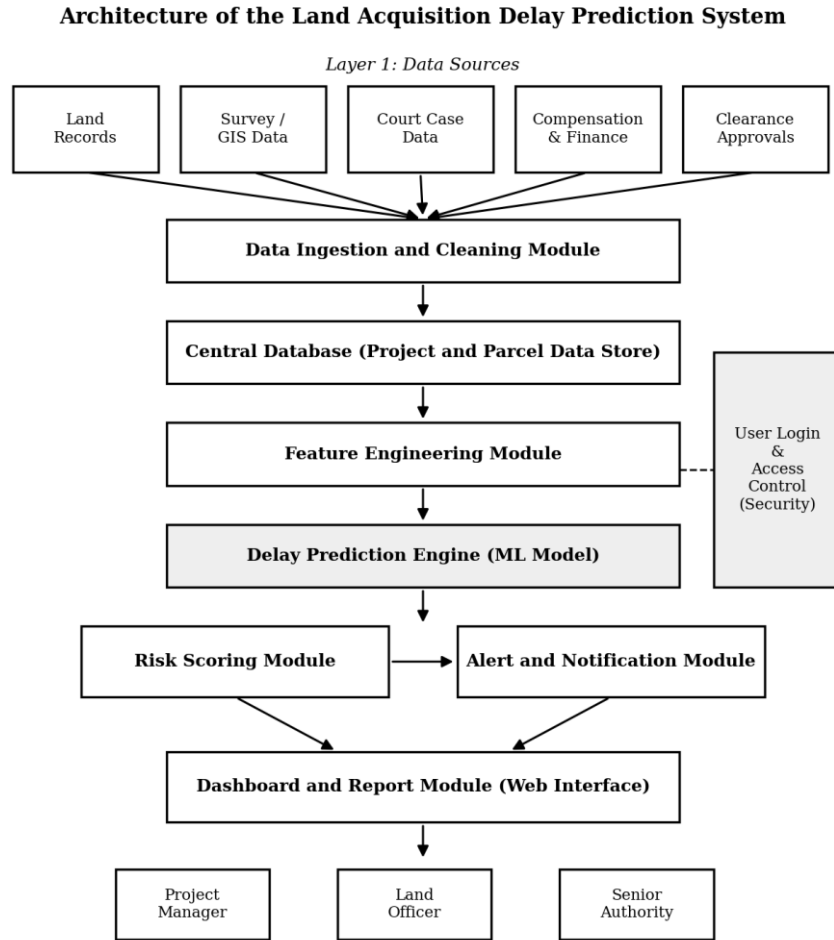


Fig 4.1: Architecture Block Diagram of the Proposed System

4.2 The major functional modules of the proposed system.

- **Data Ingestion and Cleaning Module:** Collects data from land records, survey, court, finance and approval sources and cleans it.
- **Central Database Module:** Stores parcel details, project stages, owners, cases, payments and history in a structured form.
- **Feature Engineering Module:** Creates model inputs like stage duration, dispute count, ownership complexity and approval pending days.
- **Delay Prediction Engine:** Trains on past projects and predicts the delay probability for each parcel and stage.
- **Risk Scoring Module:** Converts predictions into Low, Medium and High risk levels and lists the main reasons for the risk.
- **Alert and Notification Module:** Sends email or SMS alerts to officers when a parcel becomes high risk or a deadline is near.
- **Dashboard and Report Module:** Shows project-wise and district-wise status, risk maps, charts and downloadable reports.
- **User Management and Security Module:** Handles login, user roles and permissions.

4.3 The functional and non-functional requirements.

Functional requirements describe what the system should do:

- The system shall allow authorised users to add, update and upload parcel and project data.
- The system shall validate and clean the data before storing it.
- The system shall predict the delay risk for every parcel and every stage.
- The system shall generate alerts when the risk is high.
- The system shall provide a dashboard with filters by project, district and stage.
- The system shall generate reports in PDF and Excel format.
- The system shall allow the admin to manage users and roles.

Non-functional requirements describe how well the system should work:

Requirement	Description
Performance	Dashboard pages should load within 3 seconds and predictions should refresh at least once a day.
Scalability	Should support thousands of parcels and several projects at the same time.
Reliability	Should have at least 99% availability with regular data backups.
Security	Role-based access, encrypted passwords and data transfer, and activity logs.
Usability	Simple screens that non-technical officers can use after short training.
Maintainability	Code should be modular and documented so that changes are easy.
Accuracy	Prediction model should reach a reasonable accuracy and be retrained as new data arrives.

4.4 Explanation of how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

Modularity: The system is divided into separate modules like ingestion, prediction, alerts and dashboard. Each module has one clear job and talks to others through defined interfaces, so one part can be changed without affecting the rest.

Scalability: The layered design and a database that can grow allow more projects, districts and users to be added. The prediction part can be run on a bigger server or in the cloud when data increases.

Maintainability: Because of the modular structure, bugs are easy to find and fix. The model can be retrained or replaced without changing the dashboard, and proper documentation and version control help new developers understand the code.

Reliability: Data validation, regular backups, error handling and logging make sure the system keeps working and does not lose data. Alerts are re-sent if the first attempt fails.

Usability: The dashboard uses simple colours (green, yellow, red) for risk, easy filters and clear reports, so that officers with little technical knowledge can understand it quickly.

Security: Land records are sensitive, so the system uses login with role-based access, encryption of data and passwords, and audit logs to track who changed what.

5. Justification of the Chosen Methodology/Framework

5.1 Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).

For this project, the Agile methodology using the Scrum framework is selected. The work will be divided into sprints of two to three weeks, and at the end of each sprint a working part of the system will be delivered and shown to the users.

A possible sprint plan is given below.

Sprint	Work Planned
Sprint 1	Requirement gathering, database design and data ingestion module.
Sprint 2	Data cleaning, feature engineering and basic dashboard.
Sprint 3	First version of the delay prediction model and risk scoring.
Sprint 4	Alert module, reports and user management.
Sprint 5	Testing with real sample data, security checks, deployment and user training.

5.2 Justify why the selected methodology/framework is suitable for the proposed system.

Waterfall would need all requirements to be fixed at the start, but in this project the requirements are not completely clear. Officers will understand what they really need only after seeing the first predictions and dashboards. Agile allows such changes without a big cost.

The prediction model also needs experiments. The first model may not be accurate, so it has to be tested, improved and tested again with better data. The short sprint cycles of Scrum fit this trial-and-improve nature well. Another reason is that many stakeholders are involved. Regular sprint reviews let officers, project managers and legal staff give feedback, and the product owner can set priorities. Spiral model was also considered, but its risk analysis cycles are heavy for a small team, while Scrum gives similar flexibility with less overhead.

5.3 Explanation of how it supports successful project development and maintenance.

In development, Scrum gives early working versions, so users can see the progress and the team can correct mistakes early. Daily stand-up meetings and sprint retrospectives keep the team in sync and help solve problems quickly. The prioritised backlog makes sure the most important features (such as data integration and risk prediction) are built first.

In maintenance, the same cycle can continue after the release. New requirements, bug fixes and model retraining are added to the backlog and delivered in small updates. This keeps the system useful as policies, data and user needs change over time.

6. Expected Outcomes and Limitations

6.1 The expected benefits of the proposed solution.

- Delays are identified early, so officers get more time to solve them.
- Officers can focus on high-risk parcels instead of spreading effort equally.
- Project cost overruns and contractor claims are reduced.
- All departments work from one common and updated set of data.
- Senior authorities get clear and real-time visibility of progress.
- Decisions are based on data and past trends instead of only experience.
- Land owners benefit from faster processing and better transparency.

6.2 Explanation of how the solution addresses the identified challenges.

Each challenge identified in section 2 is handled by a part of the system. The problem of scattered data is solved by the central database and ingestion module. Poor data quality is reduced by validation and cleaning rules. Late detection of problems is solved by the prediction engine and the alert module, which warn officers before the deadline is missed. Delayed approvals and compensation disputes are tracked as risk factors, so the officers can see them in advance and take action. Finally, the dashboard gives senior authorities the transparent view which was missing earlier, and this helps in controlling cost overruns.

6.3 Potential risks, implementation challenges, and limitations.

There are also some risks and limitations to be kept in mind:

- **Data quality risk:** If past data is incomplete or wrong, the predictions will also be weak.
- **User acceptance:** Officers may resist changing from their existing manual way of working.
- **Integration problems:** Connecting with old government systems may be difficult.
- **Security and privacy:** Land data is sensitive and needs strong protection and legal compliance.
- **Prediction limits:** The model gives only a probability, not a certainty, and cannot predict sudden events such as a policy change or a court order.
- **Cost and skills:** Training, hardware and maintenance need budget and skilled people.

6.4 Suggestion of possible future enhancements.

- Adding satellite images and GIS maps to track land use and encroachment automatically.

- Giving recommendations on what action to take to remove each delay.
- A mobile app for field officers to update data on site.
- A portal or SMS service for land owners to check their case and compensation status.
- Using advanced models and text analysis of court documents to improve accuracy.
- Extending the system to other types of projects and other states.

In conclusion, the proposed predictive analytics system changes the land acquisition process from a reactive one to a proactive one. If it is built using the Scrum approach and good software engineering practices, it can help the government finish infrastructure projects on time and with less cost.